

Spring MVC

Spring MVC is module of spring framework which is used for develop the web application by using spring framework like as servlet and jsp using MVC approach.

What is Spring MVC?

Spring MVC is a Java-based framework that is mostly used for developing web applications. It follows the MVC (Model-View-Controller) Design Pattern. This design pattern specifies that an application consists of a data model, presentation information, and control information.

This framework is developed around a DispatcherServlet which dispatches requests to handlers.

Spring Model-View-Controller

- **Model** – A model contains the application's data. A data set might be a single object or a group of things.
- **Controller** – A controller houses an application's business logic. The @Controller annotation is used here to identify the class as the controller.
- **View** – A view is a representation of the delivered information in a certain format. In most cases, JSP+JSTL is utilized to construct a view page. Spring does, however, support additional view technologies such as Apache Velocity, Thymeleaf, and FreeMarker.
- **Front Controller** – The DispatcherServlet class serves as the front controller in Spring Web MVC. It is in charge of managing the flow of the Spring MVC application.

Q. What is MVC?

MVC stands for model view and controller basically it is design pattern in web application development or it is standard in web application development where we can separate the designing layer/logics and business logics/layer called as MVC.

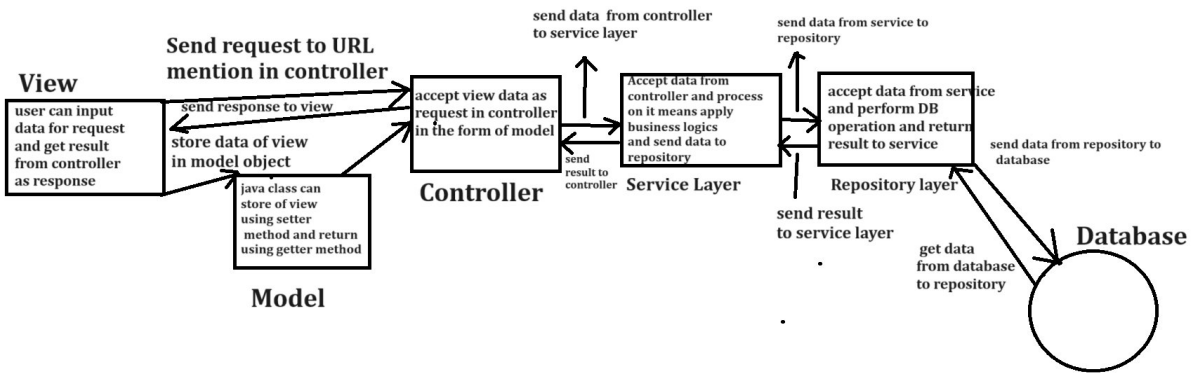
M: M stands for model it is java POJO class which is used for to store data send by view and pass to controller

V: V stands for view it is designing layer or UI Part of application and it used for accept the input data send by end user and generate result as response on view page and normally we can design view using HTML/CSS/.JSP / React / Angular etc.

C : Controller is a class in web application which help us to accept the data send by view through the model and process on it and send to service layer for business logic execution purpose and get result from service and send to view page as response.

Controller can design using servlet or in the spring MVC @Controller annotation.

When we work with MVC architecture then your code flow like as



Q. Why use Spring MVC if we have Servlet and JSP already?

1. If we use the Servlet and JSP then we need to implement the manually MVC architecture and it is very complicated and time consuming process
 2. If we use JSP for develop the web application then we need to write designing and business logics at same place so it very complicated when we want to modify the code in future
 3. Servlet and JSP generate the boiler plate code
- Etc.

Note: these are the some reason we use the Spring MVC for develop the web application except servlet and jsp.

Advantages of Spring MVC Framework

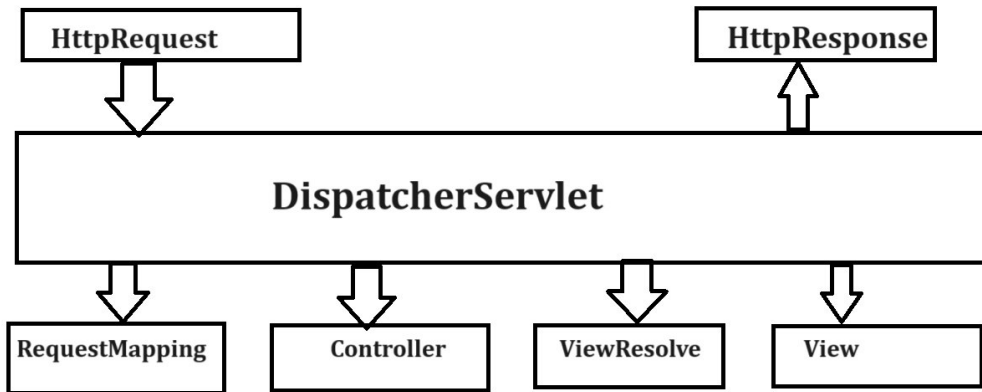
- The container is used for the development and deployment of applications and uses a lightweight servlet.
- It enables rapid and parallel development.
- Development of the application becomes fast.
- Easy for multiple developers to work together.
- Easier to Update the application.
- It is Easier to Debug because we have multiple levels in the application.

Disadvantages of Spring MVC Framework

- It has high complexity to develop the applications using this pattern.
- It is not suitable for small applications which affect the application's performance and design.

Spring MVC Framework follows the **Model-View-Controller** architectural design pattern which works around the Front Controller i.e. the **Dispatcher Servlet**. The Dispatcher Servlet handles and dispatches all the incoming **HTTP requests** to the appropriate controller. It uses **@Controller** and **@RequestMapping** as default request handlers. The **@Controller** annotation defines that a particular class is a controller. **@RequestMapping** annotation maps web requests to Spring Controller methods.

If you want to work with Spring MVC we need to know the Spring MVC architecture



Now we want to discuss about Architecture practically.

HttpRequest: http request can happen when user runs the web application or when user click on hyperlink or when user submit form or when user refresh web page or visit any website.

DispatcherServlet: basically DispatcherServlet is inbuilt class present in spring MVC library from package `org.springframework.web.servlet.DispatcherServlet` and this class work as front controller means all request map by internally using DispatcherServlet means this class internally manage which view request send to which controller etc and it is configured in web.xml file.
DispatcherServlet configured like as

web.xml

```
<servlet>
  <servlet-name>spring</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>spring</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```

Note: here / url pattern means you can call your home page of application as root request when we run your web application.

Controller: Controller is class in spring MVC mark with `@Controller` annotation and it is used for accept requested data using view layer or view page.

Example:

```
@Controller
public class TestCtrl{
}
```

RequestMapping: RequestMapping is function defined under controller class and mark with @RequestMapping annotation and this function work as requested URL in spring MVC means we map the URL at server side in controller and execute the logic at server side on client request and generate response send to view page on client side.

```
@Controller
public class TestCtrl
{ @RequestMapping("/",method=RequestMethod.GET | RequestMethod.POST)
  public String homePage(){
    return "home";//home.jsp view page
  }
  @RequestMapping("/welcome")
  public String welcomePage(){
    return "welcome";//internally welcome.jsp page
  }
}
```

ViewResolver: ViewResolver is XML file or it may java class and which is used for set the location of view pages in application as well as set the extension of view page and redirects the view from controller and if we want to manage the ViewResolver we required to use org.springframework.web.servlet.view.InternalResourceViewResolver class

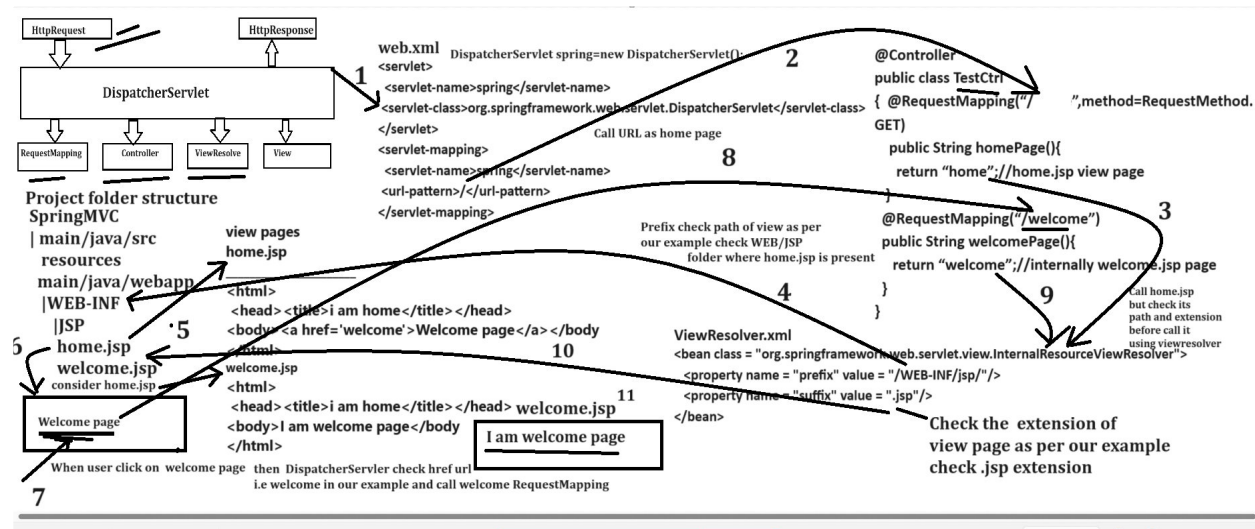
You can write view resolver using XML and for that we required to write following tag in XML file.

```
<bean class = "org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name = "prefix" value = "/WEB-INF/jsp/" />
  <property name = "suffix" value = ".jsp" />
</bean>
```

ViewResolver using annotation class

```
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.raven.webmvcapp")
public class ApplicationConfiguration {
    @Bean
    public ViewResolver viewResolver() {
        InternalResourceViewResolver resourceViewResolver = new
        InternalResourceViewResolver();
        resourceViewResolver.setPrefix("/WEB-INF/view/");
        resourceViewResolver.setSuffix(".jsp");
        return resourceViewResolver;
    }
}
```

View: View page is your JSP page or any another type of page where user can see the results as response and send request and normally in spring MVC we design view pages using .jsp extension



How to implement spring MVC by using eclipse

1. Create Dynamic web project

2. Convert normal dynamic web project in to maven

right click on project --- select configure option --- convert to maven project

3. Add following maven dependencies

- spring core
- spring context.
- spring mvc
- servlet
- jsp
- jstl

```

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>5.3.17</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.17</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
<dependency>

```

```

    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.17</version>
</dependency>
<!-- https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
    <scope>provided</scope>
</dependency>
<!-- https://mvnrepository.com/artifact/javax.servlet.jsp/javax.servlet.jsp-api -->
<dependency>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>javax.servlet.jsp-api</artifactId>
    <version>2.3.3</version>
    <scope>provided</scope>
</dependency>
<!-- https://mvnrepository.com/artifact/javax.servlet/jstl -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
</dependency>

```

4. Configure Dispatcher Servlet

We need to configure dispatcher servlet in web.xml file

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee" xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" id="WebApp_ID" version="4.0">
    <servlet>
        <servlet-name>spring</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>spring</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>

```

</web-app>

5. Create ViewResolver and set path of view and suffix of view

Note: right click on project ---new -- select folder option -- give folder name like as src/main/resources and create viewresolver.xml in that folder.

if we want to create ViewResolver XML file you have to create src/main/resources folder

viewresolver.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:beans="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-package="org.techhub" />
    <mvc:annotation-driven />
    <!-- ===== -->
    <!-- 2. view resolver for JSP -->
    <!-- ===== -->
    <bean id="viewResolver"
          class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/JSP/" />
        <property name="suffix" value=".jsp" />
    </bean>
</beans>
```

Once we create viewresolver.xml we required to set it as init-param in web.xml file under DispatcherServlet shown in following code.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xmlns="http://xmlns.jcp.org/xml/ns/javaee"
```

```

        xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
        id="WebApp_ID" version="4.0">
        <servlet>
            <servlet-name>spring</servlet-name>
            <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
            <init-param>
                <param-name>contextConfigLocation</param-name>
                <param-value>classpath:viewresolver.xml</param-value>
            </init-param>
            <load-on-startup>1</load-on-startup>
        </servlet>
        <servlet-mapping>
            <servlet-name>spring</servlet-name>
            <url-pattern>/</url-pattern>
        </servlet-mapping>
    </web-app>

```

6. Create View page

Create view pages under WEB-INF/JSP folder

home.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
    <a href="welcome">Welcome page</a>
</body>
</html>

```

welcome.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">

```



```
<title>Insert title here</title>
</head>
<body>
  <h1>Welcome page</h1>
</body>
</html>
```

7. Create Controller and request mapping
you need to create controller we under src/main/java folder

package org.techhub;

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
@Controller
public class TestCtrl {
    @RequestMapping("/")
    public String homePage() {
        return "home";
    }
    @RequestMapping("/welcome")
    public String welcomePage() {
        return "welcome";
    }
}
```

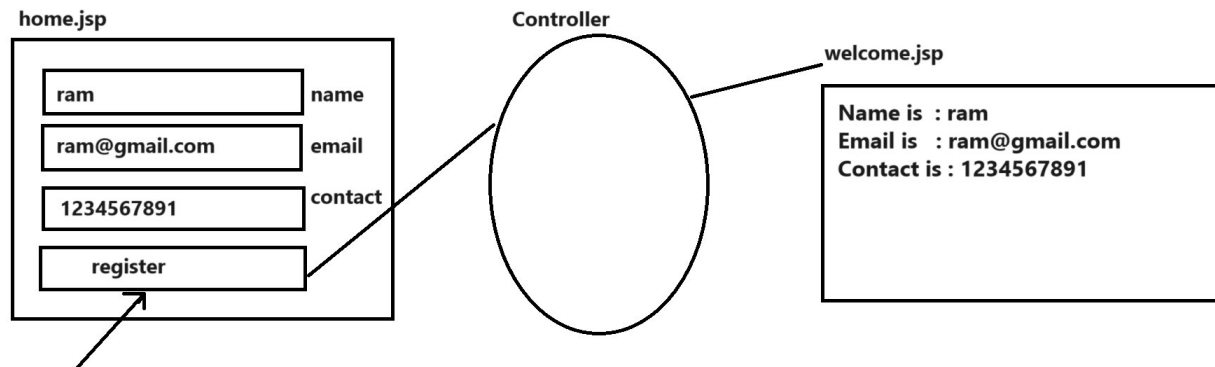
8. Run Project on server.

How to submit HTML form to controller using Spring MVC

If we want to submit HTML form to controller using spring MVC we have following steps.

If we want to submit the HTML form in spring MVC we have use following steps.

1. Create Project
2. Create view page.
3. Create Controller with request mapping and pass HttpServletRequest as parameter in RequestMapping
4. Accept form data using HttpServletRequest interface
5. Pass Map as parameter in RequestMapping where HttpServletRequest is parameter
6. Store requested data in Map.
7. Access Map data on view page using Expression Language.



If we think about above diagram we want to design registration page using spring MVC and submit to controller and catch view page data in controller and send to welcome.jsp page.

1. Create Maven project using

2. Design form or HTML form using .jsp or view page using .jsp

home.jsp

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">

<html>
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Home</title>
    <style>
    input{
        width:400px;
        height:40px;
    }
    </style>
</head>
<body>
<form name='frm' action='save' method='POST'>
    <input type='text' name='name' value='/'><br><br>
    <input type='text' name='email' value='/'><br><br>
    <input type='text' name='contact' value='/'><br><br>
    <input type='submit' name='s' value='Register'><br><br>
</form>

</body>
</html>
  
```

3. Design URL or request mapping in controller where we want to submit form

HomeController.java

```
package org.techhub.FormSubUsingMVCAPP.controller;
import java.io.IOException;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class HomeController {
    @RequestMapping(value="/")
    public String test(HttpServletResponse response) throws IOException{
        return "home";
    }
    @RequestMapping("/save")
    public String acceptFormData(HttpServletRequest request, Map map) {
        String name=request.getParameter("name");
        String email=request.getParameter("email");
        String contact=request.getParameter("contact");
        map.put("n",name);
        map.put("e", email);
        map.put("c", contact);
        return "welcome";
    }
}
```



```

    input{
        width:400px;
        height:40px;
    }
</style>
</head>
<body>
<form name='frm' action='save' method='POST'>
    <input type='text' name='name' value=''/><br><br>
    <input type='text' name='email' value=''/><br><br>
    <input type='text' name='contact' value=''/><br><br>
    <input type='submit' name='s' value='Register'/><br><br>
</form>
</body>
</html>

```

2. Create Model class e.g Register.java

```

package org.techhub.FormSubUsingMVCAPP.controller;

public class Register {
    private String name;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getContact() {
        return contact;
    }
    public void setContact(String contact) {
        this.contact = contact;
    }
    private String email;
    private String contact;
}

```

3. Create Controller and pass model as parameter in RequestMapping where we want to submit data

```
package org.techhub.FormSubUsingMVCAPP.controller;
import java.io.IOException;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class HomeController {
    @RequestMapping(value="/")
    public String test(HttpServletResponse response) throws IOException{
        return "home";
    }
    @RequestMapping("/save")
    public String acceptFormData(Register register, Map map) {
        map.put("r", register);
        return "welcome";
    }
}
```

4) Create Destination page for view data e.g. welcome.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<h1>Name is &nbsp; ${r.getName()}</h1>
<h1>Email is &nbsp; ${r.getEmail()}</h1>
<h1>Contact is &nbsp; ${r.getContact()}</h1>
</body>
</html>
```



if we think about above diagram we have controller name as HomeController and in HomeController we have URL name as RequestMapping("/") means this RequestMapping call you home page i.e home.jsp and when user click on register button then we call RequestMapping("/save") URL from controller and in RequestMapping function we have parameters one Register register as model and one is Map interface so spring framework internally accept all view data and store in model class means spring container internally map the view page control name and model class variable name and all setter method according to that and store all view page in register model object shown in diagram and we have one more parameter name as Map and we have statement in function `map.put("r", register)` here r work as reference of Register model class and we pass r on welcome.jsp so we have statement on welcome.jsp is `Name is ${r.getName()}` this statement call `getName()` method of register model class `r.getEmail()` call register `getEmail()` and so on means r points from welcome.jsp to register which we create in HomeController RequestMapping.

How to Connect Spring MVC Application with Database using Spring JDBC

1. Create Maven Project using co.ntier archetype
2. Add the Following maven dependencies in pom.xml file

1. Spring MVC

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>3.1.0.RELEASE</version>
</dependency>
<dependency>
```

2. Spring Core

```
<dependency>
    <groupId>org.springframework</groupId>
```

```

        <artifactId>spring-core</artifactId>
        <version>3.1.0.RELEASE</version>
    </dependency>

```

3. Spring Context

```

    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-core</artifactId>
        <version>3.1.0.RELEASE</version>
    </dependency>

```

4. Spring JDBC

```

<!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>3.0.1.RELEASE</version>
</dependency>

```

5. Mysql Connector

```

<!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.28</version>
</dependency>

```

6. servlet dependency

7. JSP and JSTL dependency

```

    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>servlet-api</artifactId>
        <version>2.4</version>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>javax.servlet.jsp</groupId>
        <artifactId>jsp-api</artifactId>
        <version>2.1</version>
        <scope>provided</scope>
    </dependency>

```

2. Open the MVCConfiguration.java class or config class and configure bean for following DriverManagerDataSource and JdbcTemplate


```
package org.techhub.SpringMVCWithDatabase.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurerAdapter;
import org.springframework.web.servlet.view.InternalResourceViewResolver;
```

```
@Configuration
```

```
@ComponentScan(basePackages="org.techhub.SpringMVCWithDatabase")
```

```
@EnableWebMvc
```

```
public class MvcConfiguration extends WebMvcConfigurerAdapter{
```

```
    @Bean
```

```
    public ViewResolver getViewResolver(){
```

```
        InternalResourceViewResolver resolver = new InternalResourceViewResolver();
```

```
        resolver.setPrefix("/WEB-INF/views/");
```

```
        resolver.setSuffix(".jsp");
```

```
        return resolver;
```

```
    }
```

```
    @Override
```

```
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
```

```
        registry.addResourceHandler("/resources/**").addResourceLocations("/resources/");
```

```
    }
```

```
    @Bean(name="dataSource")
```

```
    public DriverManagerDataSource getDataSource() {
```

```
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
```

```
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
```

```
        dataSource.setUsername("root");
```

```
        dataSource.setPassword("root");
```

```
        dataSource.setUrl("jdbc:mysql://localhost:3306/mysql");
```

```
        return dataSource;
```

```
    }
```

```
    @Bean(name="template")
```

```
    public JdbcTemplate getTemplate() {
```

```
        return new JdbcTemplate(getDataSource());
```

```
    }
```

}

3) Design view page

Note: we want to design view page with field name, email and contact so you have to open views folder under webapp -WEB-INF -views

home.jsp

```
1 <%@page contentType="text/html" pageEncoding="UTF-8"%>
2 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
3   "http://www.w3.org/TR/html4/loose.dtd">
4 <html>
5   <head>
6     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7     <title>Home</title>
8     <style>
9       input{
10         width:400px;
11         height:40px;
12         border-radius: 5px;
13       }
14     </style>
15   </head>
16   <body>
17     <input type='text' name='name' value=''/><br><br>
18     <input type='text' name='email' value=''/><br><br>
19     <input type='text' name='contact' value=''/><br><br>
20     <input type='submit' name='s' value='Register'/><br><br>
21   </body>
22 </html>
23
```



http://localhost:9090/SpringMVCWithDatabase/

safd

asdf

afas

Register

HomeController.java

```

1 package org.techhub.SpringMVCWithDatabase.controller;
2 import java.io.IOException;
3 import javax.servlet.http.HttpServletResponse;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.web.bind.annotation.RequestMapping;
6 import org.springframework.web.servlet.ModelAndView;
7 @Controller
8 public class HomeController {
9     @RequestMapping(value="/")
10    public String test(HttpServletResponse response) throws IOException{
11        return "home"; //home.jsp
12    }
13 }
14

```

4) Create Model for view page

```

1 package org.techhub.SpringMVCWithDatabase.model;
2 public class Register {
3     private String name;
4     public String getName() {
5         return name;
6     }
7     public void setName(String name) {
8         this.name = name;
9     }
10    public String getEmail() {
11        return email;
12    }
13    public void setEmail(String email) {}
14    public String getContact() {
15        return contact;
16    }
17    public void setContact(String contact) {
18        this.contact = contact;
19    }
20    private String email;
21    private String contact;
22 }
23
24
25

```

5. Create RequestMapping in controller and submit view page and accept view data in model.

```

1 <%@page contentType="text/html" pageEncoding="UTF-8"%>
2 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
3 "http://www.w3.org/TR/html4/loose.dtd">
4 <html>
5 <head>
6 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7 <title>Home</title>
8 <style>
9 input{
10 width:400px;
11 height:40px;
12 border-radius: 5px;
13 }
14 </style>
15 </head>
16 <body>
17 <form name='frm' action='save' method='POST'>
18 <input type='text' name='name' value='' /><br><br>
19 <input type='text' name='email' value='' /><br><br>
20 <input type='text' name='contact' value='' /><br><br>
21 <input type='submit' name='s' value='Register' /><br><br>
22 </form>
23 </body>
24 </html>
25

```



```

11 @Controller
12 public class HomeController {
13     @RequestMapping(value="/")
14     public String test(HttpServletResponse response) throws IOException{
15         return "home"; //home.jsp
16     }
17     @RequestMapping(value="/save",method=RequestMethod.POST)
18     public String saveData(Register model ,Map map) {
19         return "home";
20     }
21 }
22

```

```

class Register
{ private String name;
  private String email;
  private String contact;
  //setter and getter method
}

```



6. Create Service layer and create its object in Controller using Autowired annotation and pass model object from controller to Service

If we want to create service layer we have inbuilt annotation name as @Service

Syntax:

```

@Service("refname")
public class classname{
}

```

Example:

```

@Service("regService")
public class RegService
{
    public boolean isRegister(Register register)
    { //
    }
}

```

Controller

```

public class HomeController{
    @Autowired
    RegService regService; //create object of Service class
}

```

7. Create Repository class and create object of JdbcTemplate in Repository using @Autowired annotation

If we want to create repository class we have @Repository annotation

Syntax:

```
@Repository("refname")
```

```
public class classname{  
}
```

Example:

```
@Repository("regRepo")  
public class RegRepository{  
    @Autowired  
    JdbcTemplate template;  
}
```

Note: you need to create object of repository class in service layer class using @Autowired annotation and call repository methods in service class.

The screenshot illustrates the integration of a repository class into a Spring MVC application. It shows a web form for registration, a `Register` class with attributes, a `RegService` class that uses `RegRepository` and `JdbcTemplate`, and an `MvcConfiguration` class that configures the database and repository beans. Arrows highlight the flow of data from the form to the service layer and the dependency on the repository.

```
<form name='frm' action='save' method='POST'>  
  <input type='text' value='shyam' /> name  
  <input type='text' value='shyam@gmail.com' /> email  
  <input type='text' value='9988776655' /> contact  
  <input type='button' value='Register' />  
</form>  
  
class Register{  
  private String name;  
  private String email;  
  private String contact;  
} //setter and //getter methods  
  
@Service("regService")  
public class RegService {  
  @Autowired  
  RegRepository regRepo;  
  public boolean isRegister(Register register) {  
    boolean b = regRepo.isRegister(register);  
    return b;  
  }  
  
  @Repository("regRepo")  
  public class RegRepository {  
    @Autowired  
    JdbcTemplate template;  
    public boolean isRegister(final Register register) {  
      int value = template.update("insert into register values(?,?,?)", new PreparedStatementSetter() {  
        @Override  
        public void setValues(PreparedStatement ps) throws SQLException {  
          ps.setString(1, register.getName());  
          ps.setString(2, register.getEmail());  
          ps.setString(3, register.getContact());  
        }  
      });  
      return value > 0 ? true : false;  
    }  
  }  
}
```

```
@Configuration  
@ComponentScan(basePackages="org.techhub.SpringMVCWithDatabase")  
@EnableWebMvc  
public class MvcConfiguration extends WebMvcConfigurerAdapter{  
  @Bean(name="dataSource")  
  public DriverManagerDataSource getDataSource() {  
    DriverManagerDataSource dataSource = new DriverManagerDataSource();  
    dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");  
    dataSource.setUsername("root");  
    dataSource.setPassword("root");  
    dataSource.setUrl("jdbc:mysql://localhost:3306/mysql");  
    return dataSource;  
  }  
  @Bean(name="template")  
  public JdbcTemplate getTemplate() {  
    return new JdbcTemplate(getDataSource());  
  }  
}
```

Final Repository class

```
package org.techhub.SpringMVCWithDatabase.repository;
```

```
import java.sql.PreparedStatement;
```

```
import java.sql.SQLException;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.jdbc.core.JdbcTemplate;
```

```
import org.springframework.jdbc.core.PreparedStatementSetter;
```

```
import org.springframework.stereotype.Repository;
```

```

import org.techhub.SpringMVCWithDatabase.model.Register;

@Repository("regRepo")
public class RegRepository {
    @Autowired
    JdbcTemplate template;
    public boolean isRegister(final Register register) {
        int value = template.update("insert into register values(?,?,?)", new
PreparedStatementSetter() {
            @Override
            public void setValues(PreparedStatement ps) throws SQLException {
                ps.setString(1, register.getName());
                ps.setString(2, register.getEmail());
                ps.setString(3, register.getContact());
            }
        });
        return value > 0 ? true : false;
    }
}

```

Example: Now we want to create simple CRUD Application by using Spring MVC

Before CRUD Application we want to see how to apply or use external CSS in Spring MVC

Steps

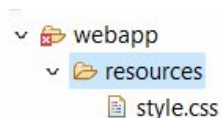
1. Configure bean for ViewResolver in configuration class and configure the Prefix and suffix means set folder path as well as extension of file

```

17 public class MvcConfiguration extends WebMvcConfigurerAdapter{
18     @Bean
19     public ViewResolver getViewResolver(){
20         InternalResourceViewResolver resolver = new InternalResourceViewResolver();
21         resolver.setPrefix("/WEB-INF/views/");
22         resolver.setSuffix(".jsp");
23         return resolver;
24     }
}

```

2. Open resources folder under webapp and create .css file

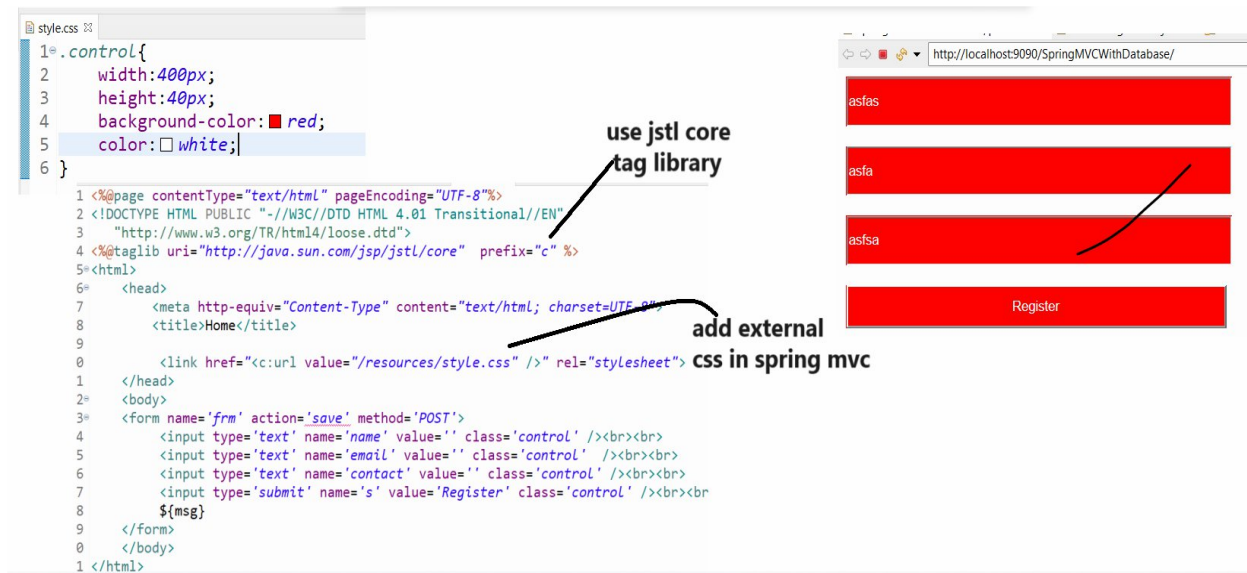


3. Use external CSS using JSTL core tag library

Before that you have to add JSTL Maven dependency

```
<!-- https://mvnrepository.com/artifact/javax.servlet/jstl -->
```

```
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>
```



CRUD Application using Spring MVC

Just we want to design one employee form with field id, name, contact and salary and perform Insert, Delete, Update, and Select and search operation on Employee form using Spring MVC.

Front End Technology: HTML, CSS, JS, BootStrap, JSP

Backend - spring, Spring MVC, Spring JDBC

Database - MYSQL

Steps to Develop the Application

1. Create Maven Project

2. Add Following Maven dependencies

a) spring jdbc b) spring core c) spring context d) spring MVC e)JSTL f) servlet g)JSP

```
<!-- Servlet Spec -->
```

```
<dependency>
```

```
  <groupId>javax.servlet</groupId>
```

```
  <artifactId>servlet-api</artifactId>
```

```
  <version>2.4</version>
```

```
  <scope>provided</scope>
```

```

</dependency>
<dependency>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>jsp-api</artifactId>
    <version>2.1</version>
    <scope>provided</scope>
</dependency>

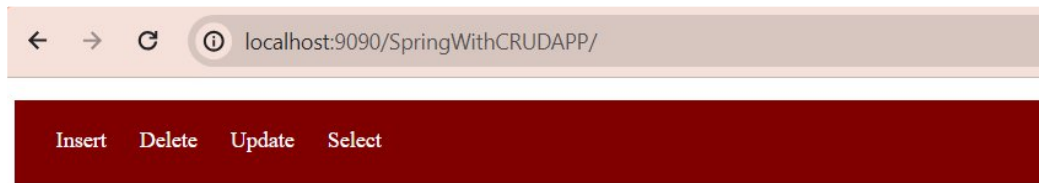
<!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>3.0.1.RELEASE</version>
</dependency>
<!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.28</version>
</dependency>
<!-- Servlet Spec -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>servlet-api</artifactId>
    <version>2.4</version>
    <scope>provided</scope>
</dependency>
<dependency>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>jsp-api</artifactId>
    <version>2.1</version>
    <scope>provided</scope>
</dependency>

<!-- https://mvnrepository.com/artifact/javax.servlet/jstl -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
</dependency>

```

3) Design Navigation bar with menu Insert, Delete, Update & select option

Note: Here use external CSS and external JS



Source code of above screen short.

style.css

```
ul{ width:100%;
    height:20px;
    background-color:maroon;
    padding:20px;
}
ul li{
    display:inline;
    padding:10px;
}
ul li a{
    text-decoration: none;
    color:white;
}
```

home.jsp

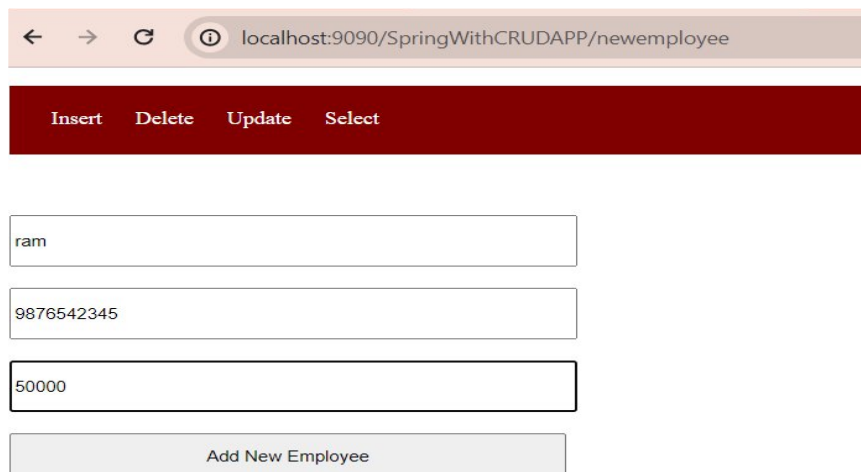
```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Home</title>
    <link href="<c:url value="/resources/CSS/style.css" />" rel="stylesheet">
  </head>
  <body>
    <ul>
      <li><a href=" ">Insert</a></li>
      <li><a href=" ">Delete</a></li>
      <li><a href=" ">Update</a></li>
      <li><a href=" ">Select</a></li>
```

```
</ul>
</body>
</html>
```

HomeController.java

```
package org.techhub.SpringWithCRUDAPP.controller;
import java.io.IOException;
import javax.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class HomeController {
    @RequestMapping(value="/")
    public String test(HttpServletResponse response) throws IOException{
        return "home";
    }
}
```

Once we design navigation bar you have to design view page for Employee insert data using field name , contact and salary so we want to create file name as saveemployee.jsp as view page and call this file using RequestMapping url name as newemployee and pass this newemployee url in hyperlink insert menubar.



Source code of above screen short

style.css

```
.control{
    width:400px;
```

```

        height:40px;
    }
HomeController.java
package org.techhub.SpringWithCRUDAPP.controller;
import java.io.IOException;
import javax.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class HomeController {
    @RequestMapping("/newemployee")
    public String saveEmployee() {
        return "saveemployee";
    }
}

home.jsp (update insert link code <a href='newemployee'>) only
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Home</title>
        <link href="<c:url value="/resources/CSS/style.css"/>" rel="stylesheet">
    </head>
    <body>
        <ul>
            <li><a href='newemployee'>Insert</a></li>
            <li><a href=''>Delete</a></li>
            <li><a href=''>Update</a></li>
            <li><a href=''>Select</a></li>
        </ul>
    </body>
</html>

saveemployee.jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>

```

```

<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<link href="<c:url value="/resources/CSS/style.css" />" rel="stylesheet">
</head>
<body>
<jsp:include page="home.jsp"></jsp:include>
<br><br>
<input type='text' name='name' value="" class='control' /><br><br>
<input type='text' name='contact' value="" class='control' /><br><br>
<input type='text' name='salary' value="" class='control' /><br><br>
<input type='submit' name='s' value='Add New Employee' class='control' /><br><br>
</body>
</html>

```

Note: After design this UI when user click on Add New Employee button then we want to submit view page to controller and store view data in model class and pass to service and as well pass to repository and store in database table.

For that we have to create new request mapping in controller name as saveemp and submit form to saveemp request mapping but before that we need to design model class name as Employee with four field id name contact salary and pass this model class as parameter in saveemployee request mapping function

Complete Source for Employee Record Save

Insert Delete Update Select

Add New Employee

Employee Save Success.....

```

mysql> use mysql;
Database changed
mysql> create table employeemvc(id int(5) primary key auto_increment,name varchar(200) not null, contact varchar(200) not null unique,salary int(5));
Query OK, 0 rows affected, 2 warnings (0.18 sec)

mysql>

```

style.css

```
ul{
    width:100%;
    height:20px;
    background-color:maroon;
    padding:20px;
}
ul li{
    display:inline;
    padding:10px;
}
ul li a{
    text-decoration: none;
    color:white;
}
.control{
    width:400px;
    height:40px;
}
```

home.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Home</title>
        <link href="
```

saveemployee.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<link href="<c:url value="/resources/CSS/style.css" />" rel="stylesheet">
</head>
<body>
    <jsp:include page="home.jsp"></jsp:include>
    <br>
    <br>
    <form name='frm' action='saveemp' method='POST'>
        <input type='text' name='name' value="" class='control' /><br>
        <br> <input type='text' name='contact' value="" class='control' /><br>
        <br> <input type='text' name='salary' value="" class='control' /><br>
        <br> <input type='submit' name='s' value='Add New Employee'
            class='control' /><br>
    <br>
    </form>
    ${msg}
</body>
</html>

```

Employee.java

```

package org.techhub.SpringWithCRUDAPP.model;

public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}

```

```

    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
    public String getContact() {
        return contact;
    }
    public void setContact(String contact) {
        this.contact = contact;
    }
    private int sal;
    private String contact;
}
HomeController.java
package org.techhub.SpringWithCRUDAPP.controller;

import java.io.IOException;
import java.util.Map;

import javax.servlet.http.HttpServletResponse;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import org.techhub.SpringWithCRUDAPP.service.EmployeeService;

@Controller
public class HomeController {

    @Autowired
    EmployeeService empService;
    @RequestMapping(value = "/")
    public String test(HttpServletResponse response) throws IOException {
        return "home";
    }

    @RequestMapping("/newemployee")

```

```

    public String saveEmployee() {
        return "saveemployee";
    }

    @RequestMapping("/saveemp")
    public String saveEmployee(Employee employee, Map map) {
        boolean b=empService.isSave(employee);
        if(b) {
            map.put("msg","Employee Save Success.....");
        }
        else {
            map.put("msg","Some problem is there.....");
        }
        return "saveemployee";
    }
}

```

EmployeeService.java

```

package org.techhub.SpringWithCRUDAPP.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import org.techhub.SpringWithCRUDAPP.repository.EmployeeRepository;
@Service("empService")
public class EmployeeService {
    @Autowired
    EmployeeRepository empRepo;
    public boolean isSave(Employee employee) {
        return empRepo.isSave(employee);
    }
}

```

EmployeeRepository.java

```

package org.techhub.SpringWithCRUDAPP.repository;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.stereotype.Repository;
import org.techhub.SpringWithCRUDAPP.model.Employee;
@Repository("empRepo")
public class EmployeeRepository {
    @Autowired

```



```

        JdbcTemplate template;
        public boolean isSave(final Employee employee) {
            int value = template.update("insert into employeeemvc values('0',?,?,?)", new
PreparedStatementSetter() {
                @Override
                public void setValues(PreparedStatement ps) throws SQLException {
                    ps.setString(1, employee.getName());
                    ps.setString(2, employee.getContact());
                    ps.setInt(3, employee.getSal());
                }
            });
            return value > 0 ? true : false;
        }
    }
}

MVCCConfiguration.java
package org.techhub.SpringWithCRUDAPP.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurerAdapter;
import org.springframework.web.servlet.view.InternalResourceViewResolver;
@Configuration
@ComponentScan(basePackages = "org.techhub.SpringWithCRUDAPP")
@EnableWebMvc
public class MvcConfiguration extends WebMvcConfigurerAdapter {
    @Bean
    public ViewResolver getViewResolver() {
        InternalResourceViewResolver resolver = new InternalResourceViewResolver();
        resolver.setPrefix("/WEB-INF/views/");
        resolver.setSuffix(".jsp");
        return resolver;
    }
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/resources/**").addResourceLocations("/resources/");
    }
}

```

```

    }
    @Bean(name = "dataSource")
    public DriverManagerDataSource getDataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/mysql");
        dataSource.setUsername("root");
        dataSource.setPassword("root");
        return dataSource;
    }
    @Bean(name = "template")
    public JdbcTemplate getTemplate() {
        return new JdbcTemplate(this.getDataSource());
    }
}

```

Once we insert record when user click on view menu bar on home.jsp page then we want to display all records from database table. for that you have to design viewemployee.jsp page under views folder and create one RequestMapping name as viewemp and viewemployee.jsp from this request mapping

The diagram illustrates the data flow for displaying employee records. It starts with a MySQL database query: `mysql> select * from employeeemvc;` which returns a table with 3 rows. This data is retrieved by the `EmployeeRepository` (annotated with `@Repository("empRepo")`), which calls `getAllEmployee()` on the `EmployeeService` (annotated with `@Service("empService")`). The `EmployeeService` then calls `getAllEmployee()` on the `HomeController` (annotated with `@Controller`). The `HomeController` has a `viewemp` endpoint (annotated with `@RequestMapping("viewemp")`) that calls `viewEmployee()`. The `viewEmployee()` method uses `empService.getAllEmployee()` to get the list of employees and puts it into a map. The map is then used to render the `viewemployee.jsp` page, which displays a table of employee data. The table has columns for NAME, CONTACT, SALARY, DELETE, and UPDATE. The data is as follows:

NAME	CONTACT	SALARY	DELETE	UPDATE
ram	9988776655	0	DELETE	UPDATE
ganesh	ganesh@gmail.com	20000	DELETE	UPDATE
sandeep	sandeep@gmail.com	77666	DELETE	UPDATE

The `viewemployee.jsp` page also includes a menu bar with links for INSERT, VIEW, DELETE, and UPDATE. The VIEW link is highlighted, indicating it is the active link. The `viewEmployee()` method in the `HomeController` uses a `list` of `Employee` objects to render the table. The `Employee` objects are created by the `EmployeeRepository` using the `mapRow` method, which maps the database results to `Employee` objects. The `mapRow` method is annotated with `@Override` and `public Employee mapRow(ResultSet rs, int rowNum)`. It creates a new `Employee` object, sets its `id`, `name`, `contact`, and `salary` from the `ResultSet`, and returns it. The `mapRow` method is called for each row in the `ResultSet`, and the results are returned as a `List` of `Employee` objects.

Complete source of view employee data

EmployeeRepository code

```

package org.techhub.SpringWithCRUDAPP.repository;
import java.sql.PreparedStatement;

```

```

import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import java.util.*;
@Repository("empRepo")
public class EmployeeRepository {
    @Autowired
    JdbcTemplate template;
    List<Employee> list;
    public List<Employee> getAllEmployee(){
        list=template.query("select *from employeemvc",new RowMapper<Employee>() {
            @Override
            public Employee mapRow(ResultSet rs, int rowNum) throws SQLException {
                // TODO Auto-generated method stub
                Employee e = new Employee();
                e.setId(rs.getInt(1));
                e.setName(rs.getString(2));
                e.setContact(rs.getString(3));
                e.setSal(rs.getInt(4));
                return e;
            }
        });
        return list;
    }
}

```

EmployeeService.java

```

package org.techhub.SpringWithCRUDAPP.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import org.techhub.SpringWithCRUDAPP.repository.EmployeeRepository;
import java.util.*;
@Service("empService")
public class EmployeeService {
    @Autowired

```

```

    EmployeeRepository empRepo;

    public boolean isSave(Employee employee) {
        return empRepo.isSave(employee);
    }
    public List<Employee> getAllEmployee(){
        return empRepo.getAllEmployee();
    }
}

```

HomeController.java

```

package org.techhub.SpringWithCRUDAPP.controller;
import java.util.*;
import java.io.IOException;
import java.util.Map;

import javax.servlet.http.HttpServletResponse;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import org.techhub.SpringWithCRUDAPP.service.EmployeeService;

@Controller
public class HomeController {

    @Autowired
    EmployeeService empService;
    @RequestMapping(value = "/")
    public String test(HttpServletResponse response) throws IOException {
        return "home";
    }

    @RequestMapping("viewemp")
    public String viewEmp(Map map) {
        List<Employee> list=empService.getAllEmployee();
        map.put("empList", list);
    }
}

```

```
        return "viewemployee";
    }
}
```

viewemployee.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
    <jsp:include page="home.jsp"></jsp:include>
    <br><br>
    <table border='5' width='60%'>
        <tr>
            <th>NAME</th>
            <th>CONTACT</th>
            <th>SALARY</th>
            <th>DELETE</th>
            <th>UPDATE</th>
        </tr>
        <c:forEach var="e" items="${empList}">
            <tr>
                <th>${e.getName()}</th>
                <th>${e.getContact()}</th>
                <th>${e.getSal()}</th>
                <th><a href="">DELETE</a></th>
                <th><a href="">UPDATE</a></th>
            </tr>
        </c:forEach>

    </table>
</body>
</html>
```

INSERT VIEW DELETE UPDATE

NAME	CONTACT	SALARY	DELETE	UPDATE
ram	9988776655	0	DELETE	UPDATE
ganesh	ganesh@gmail.com	20000	DELETE	UPDATE
sandeep	sandeep@gmail.com	77666	DELETE	UPDATE

Now when we click on delete hyperlink then we want to delete the particular employee or when we click on update employee then we want to change or update employee data.

so here we want to use URL Rewriting Technique means here we pass data through the URL means as per our example we hide the employee id in URL because we want to delete specified employee when we click on URL means every URL has different employee record and which is unique and as per database table we have different employee id in database table for every employee so here we hide the employee id in URL address bar.

Task to Delete employee when user click on hyperlink

1. Create new request mapping Controller name as delemp and viewemployee.jsp from it and use @RequestParam annotation for access URL data.

@RequestParam is annotation in spring MVC which help us to access URL data using its name as well as it help us access request data of form or URL using its name.

viewemployee.jsp

INSERT VIEW DELETE UPDATE

NAME	CONTACT	SALARY	DELETE	UPDATE
ram	9988776655	0	DELETE	UPDATE
ganesh	ganesh@gmail.com	20000	DELETE	UPDATE
sandeep	sandeep@gmail.com	77666	DELETE	UPDATE

```
@Controller
public class HomeController {

    @RequestMapping("/delemp")
    public String deleteEmployee(@RequestParam("employeeId") Integer empId) {
        System.out.println("Employee Id is "+empId);
        return "viewemployee";
    }
}
```

```
<table border='5' width='60%'>
  <tr>
    <th>NAME</th>
    <th>CONTACT</th>
    <th>SALARY</th>
    <th>DELETE</th>
    <th>UPDATE</th>
  </tr>
  <c:forEach var="e" items="${empList}">
    <tr>
      <td>${e.getName()}</td>
      <td>${e.getContact()}</td>
      <td>${e.getSal()}</td>
      <td><a href='delemp?employeeId=${e.getId()}'>DELETE</a></td>
      <td><a href=''>UPDATE</a></td>
    </tr>
  </c:forEach>
</table>
```

console output is

Employee Id is 1

localhost:9090/SpringWithCRUDAPP/delemp?employeeId=1

when we pass employee id from view page to controller using URL Rewriting technique and access it using @RequestParam annotation then you can pass employee id to service layer and from service you can pass it to repository layer and delete the employee record from database table and return true if

record deleted successfully otherwise return false and true and false value in controller and return viewemployee.jsp so user can see the records after delete employee

Complete Source code of Delete Record

viewemployee.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<jsp:include page="home.jsp"></jsp:include>
<br><br>
<table border='5' width='60%'>
<tr>
<th>NAME</th>
<th>CONTACT</th>
<th>SALARY</th>
<th>DELETE</th>
<th>UPDATE</th>
</tr>
<c:forEach var="e" items="${empList}">
<tr>
<th>${e.getName()}</th>
<th>${e.getContact()}</th>
<th>${e.getSal()}</th>
<th><a href='delemp?employeeId=${e.getId()}'>DELETE</a></th>
<th><a href=''>UPDATE</a></th>
</tr>
</c:forEach>
</table>
</body>
</html>
```

HomeController.java

```
@Controller
public class HomeController {
```

```

    @Autowired
    EmployeeService empService;
    @RequestMapping(value = "/")
    public String test(HttpServletResponse response) throws IOException {
        return "home";
    }
    @RequestMapping("/delemp")
    public String deleteEmployee(@RequestParam("employeeId") Integer empId, Map map) {
        boolean b=empService.isDelete(empId);
        List<Employee> list=empService.getAllEmployee();
        map.put("empList", list);
        return "viewemployee";
    }
}

```

EmployeeService.java

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import org.techhub.SpringWithCRUDAPP.repository.EmployeeRepository;
import java.util.*;
@Service("empService")
public class EmployeeService {
    @Autowired
    EmployeeRepository empRepo;
    public boolean isSave(Employee employee) {
        return empRepo.isSave(employee);
    }
    public boolean isDelete(int id) {
        return empRepo.isDelete(id);
    }
}

```

EmployeeRepository.java

```

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;

```



```

import org.techhub.SpringWithCRUDAPP.model.Employee;
import java.util.*;
@Repository("empRepo")
public class EmployeeRepository {
    @Autowired
    JdbcTemplate template;
    List<Employee> list;
    public boolean isDelete(int id) {
        try {
            int value=template.update("delete from employeemvc where id=?",new Object[] {id});
            return value>0?true:false;
        }
        catch(Exception ex) {
            System.out.println("Error is "+ex);
            return false;
        }
    }
}

```

Now for update we want to hide the name, contact ,salary and id in URL means here we want to create new request mapping name as updateEmp and in this request mapping we want to access the all data using @RequestParam annotation and create new view page name as updateemployee.jsp with three text box and one hidden field means we hide the employee id from end user from form so we store the id in hidden field and name , contact and salary we store in textboxes.

← → ↻ ⓘ localhost:9090/SpringWithCRUDAPP/updateemp?empId=3&empName=sandeep&empContact=sandeep@gmail.com&empSal=77666

INSERT VIEW DELETE UPDATE

sandeepani

sandeep@gmail.com

90000

Update Employee

INSERT VIEW DELETE UPDATE				
NAME	CONTACT	SALARY	DELETE	UPDATE
sandeepani	sandeep@gmail.com	90000	DELETE	UPDATE
gajanan	98765432345	90000	DELETE	UPDATE

Source code of update

updateemployee.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<link href="<c:url value="/resources/CSS/style.css" />" rel="stylesheet">
</head>
<body>
    <jsp:include page="home.jsp"></jsp:include>
    <br>
    <br>
    <form name='frm' action='finalupdate' method='POST'>
        <input type='hidden' name='id' value='${id}' class='control' /><br>
        <input type='text' name='name' value='${name}' class='control' /><br>
        <br> <input type='text' name='contact' value='${contact}' class='control' /><br>
        <br> <input type='text' name='sal' value='${salary}' class='control' /><br>
        <br> <input type='submit' name='s' value='Update Employee' class='control' /><br>
        <br>
    </form>
    ${msg}
```

```
</body>
</html>
</body>
</html>
```

HomeController.java

```
package org.techhub.SpringWithCRUDAPP.controller;
import java.util.*;
import java.io.IOException;
import java.util.Map;
```

```
import javax.servlet.http.HttpServletResponse;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.servlet.ModelAndView;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import org.techhub.SpringWithCRUDAPP.service.EmployeeService;
```

@Controller

```
public class HomeController {
    @Autowired
    EmployeeService empService;
    @RequestMapping(value = "/")
    public String test(HttpServletResponse response) throws IOException {
        return "home";
    }
    @RequestMapping("/updateemp")
    public String updateEmployee(@RequestParam("empId") Integer
id,@RequestParam("empName") String name,@RequestParam("empContact") String
contact,@RequestParam("empSal") Integer sal,Map map) {
        map.put("id",id);
        map.put("name",name);
        map.put("contact",contact);
        map.put("salary",sal);

        return "updateemployee";
    }
    @RequestMapping(value="/finalupdate",method=RequestMethod.POST)
```

```

        public String updateEmployee(Employee employee, Map map) {
            boolean b=this.empService.updateEmployee(employee);
            if(b) {
                List<Employee> list=empService.getAllEmployee();
                map.put("empList", list);
                return "viewemployee";
            }
            else {
                map.put("msg", "Some problem is there.....");
                return "updateemployee";
            }
        }
    }
}

```

EmployeeService.java

```

package org.techhub.SpringWithCRUDAPP.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import org.techhub.SpringWithCRUDAPP.repository.EmployeeRepository;
import java.util.*;

@Service("empService")
public class EmployeeService {
    @Autowired
    EmployeeRepository empRepo;
    public boolean updateEmployee(Employee employee) {
        return empRepo.updateEmployee(employee);
    }
}

```

EmployeeRepository.java

```

package org.techhub.SpringWithCRUDAPP.repository;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;

```

```

import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;
import org.techhub.SpringWithCRUDAPP.model.Employee;
import java.util.*;

@Repository("empRepo")
public class EmployeeRepository {

    @Autowired
    JdbcTemplate template;
    List<Employee> list;

    public boolean updateEmployee(final Employee employee) {
        int value=template.update("update employeemvc set name=?,contact=?,salary=? where
id=?",new PreparedStatementSetter() {

            @Override
            public void setValues(PreparedStatement ps) throws SQLException {
                ps.setString(1,employee.getName());
                ps.setString(2,employee.getContact());
                ps.setInt(3,employee.getSal());
                ps.setInt(4,employee.getId());
            }

        });
        return value>0?true:false;
    }
}

```

Now we want to implement the Search Operation so for search operation we want to design textbox on view employee page and when user types anything in textbox according to that data we want to search employee record.

←
↻
🔍
localhost:9090/SpringWithCRUDAPP/viewemp

INSERT
VIEW
DELETE
UPDATE

NAME	CONTACT	SALARY	DELETE	UPDATE
sandeepani	sandeep@gmail.com	90000	DELETE	UPDATE
gajanan	98765432345	90000	DELETE	UPDATE
ganesh	8888888888	70000	DELETE	UPDATE
manesh	776633333333	80000	DELETE	UPDATE

if we think about above screen when user type anything in text box of any letter or value match in name or contact then specified record should be display in table. Means here we need to perform pattern base searching. so if we want to perform this task we need to know the following things

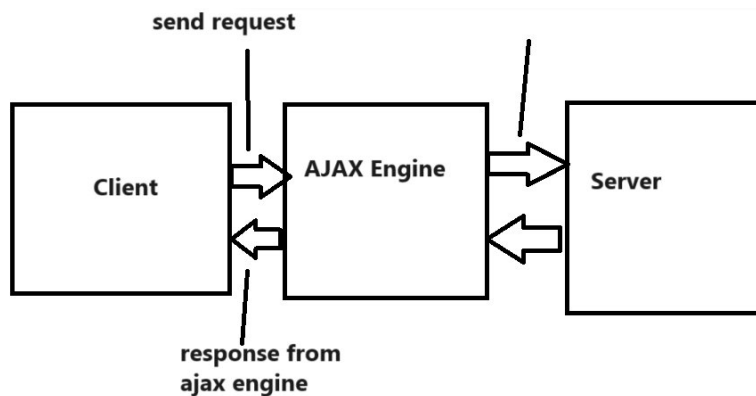
1) AJAX

2) Rest API

Q. what is AJAX?

AJAX stands for Asynchronous java script and XML and it is used for update the partial web page or performs partial updation on web page without refreshing the complete or whole web page.

so in the case of AJAX when we send request to server then direct request not send to server first request send to AJAX engine and AJAX engine send request to server and server send response to AJAX engine and then AJAX engine update on web page but not refresh complete web page just update content on web page on specified portion.



How to use AJAX Practically using JavaScript

if we want to AJAX practically using java script we have the following steps.

1) Create java scrip function

2) Write the following code.

if we want to accept request from client using AJAX we have class name as XMLHttpRequest in Javascript so we required to create its object

```
let var=new XMLHttpRequest();
```

3) you have to define onreadystatechange function XMLHttpRequest and check request status and accept response data from server in AJAX

```

let xhttp=new XMLHttpRequest();
xhttp.onreadystatechange=function(){
    if(this.readyState==4 && this.status==200){
        //accept here response from server using this.responseText
    }
};

```

Once we define onreadystatechange function you can send http request to server using specified URL for that we have the method name as open() and it is method of XMLHttpRequest object
xhttp.open("GET/POST","URL",true|false)

Open method contain three parameters.

GET/POST: this parameter decide send request to server using get or post method

URL: URL means here we specify page where we want to send request

true|false: if we set true then we send request asynchronously and if set false then send request synchronously

```

let xhttp=new XMLHttpRequest();
xhttp.onreadystatechange=function(){
    if(this.readyState==4 && this.status==200){
        //accept here response from server using this.responseText
    }
};

```

```

xhttp.open("GET","view",true);

```

once we set request method , page where we want to send request as well as true parameter or false parameter then we can send request to server and if we want to send request to server for that we have send() method of XMLHttpRequest

```

let xhttp=new XMLHttpRequest();
xhttp.onreadystatechange=function(){
    if(this.readyState==4 && this.status==200){
        //accept here response from server using this.responseText
    }
};
xhttp.open("GET","view",true);
xhttp.send();
}

```